

Research Methodology in CS (CSEG3060_3)

Lecture 5: Investigative Approaches for Solutions to Research Problems

Unit 1: Introduction to Research Problem • Mapped Course Outcome: CO2

Lecture 5: Investigative Approaches for Solutions to Research Problems

Course: Research Methodology in Computer Science (CSEG3060_3) **Program:** B.Tech (Computer Science Engineering) **Semester:** 5 **Unit:** 1 — Introduction to Research Problem **Mapped Course Outcome:** CO2 **Lecture Duration:** 60 minutes (recommended two-segment delivery)

1. Learning Objectives

By the end of this lecture, students should be able to:

1. Distinguish clearly between *identifying* a research problem and *investigating* solutions to it.
 2. Enumerate the principal investigative approaches adopted in computer science research.
 3. Match a given research problem to an appropriate investigative approach using decision criteria such as nature of evidence required, feasibility, and validity.
 4. Recognize how each investigative approach shapes data collection, instrumentation, and analysis choices.
 5. Critically evaluate the strengths and limitations of each approach in the context of computing artefacts.
 6. Justify, in writing, the selection of an investigative approach for a sample CS research problem.
-

2. Introduction

A research problem, once defined and bounded, remains an abstraction until a systematic method is identified to *investigate* possible solutions. In computer science, the phrase "investigative approach" refers to the overarching strategy that a researcher adopts to gather, analyze, and interpret evidence pertaining to a candidate solution. It is the bridge between the formulated problem (covered in earlier lectures of this unit) and the operationalized study that follows.

This is particularly important in computer science because the discipline is simultaneously:

- **Empirical** — software artefacts must be measured for performance, usability, or reliability.
- **Constructive** — new algorithms, protocols, and systems are *built*, not merely observed.
- **Formal** — many claims demand deductive proofs of correctness, complexity, or security.

Because of this triadic nature, there is no single "best" investigative approach. Instead, the researcher selects an approach that aligns the **epistemological position** (what counts as knowledge) with the **pragmatic constraints** (what can be measured or built) of the problem at hand. The remainder of this lecture lays out the available approaches, the conditions under which each is appropriate, and the way each approach constrains downstream choices in data collection, instrumentation, and interpretation.

3. Key Concepts and Terminology

Term	Definition
Investigative Approach	The overall strategy used to gather and analyze evidence about a proposed solution to a research problem.
Methodology	The broader theoretical framework informing the choice of approach; the <i>logic</i> behind the strategy.
Method	The specific techniques (e.g., surveys, controlled experiments) used to operationalize the approach.
Construct	An artefact, model, hypothesis, or theoretical statement that the research seeks to evaluate or improve.
Validity	The degree to which the evidence supports the claims made by the researcher (internal, external, construct, conclusion).
Reliability	The consistency of evidence across repetitions, observers, or instruments.
Instrumentation	The apparatus, software, or measurement scheme used to capture data.
Iteration	A cyclical refinement process wherein each investigative pass informs the next, common in design research.

4. Typology of Investigative Approaches

The approaches discussed below are not mutually exclusive; in contemporary CS research it is common to combine them as **mixed-methods** investigations. Nonetheless, every research design privileges one or two of these as the primary engine of evidence.

4.1 Experimental Investigation

Definition. An experimental investigation manipulates one or more independent variables, controls extraneous factors, and measures the effect on dependent variables using quantitative metrics.

Where it fits in CS. This is the dominant approach for evaluating algorithms, systems, and software products under controlled conditions.

Core Structure.

1. **Hypothesis statement.** H_0 : no difference; H_1 : a statistically significant difference exists.
2. **Variable identification.** Independent (e.g., algorithm variant), dependent (e.g., runtime), controlled (e.g., hardware, dataset).
3. **Experimental design.** Randomized block, factorial, Latin square, or repeated-measures designs may be used.
4. **Measurement.** Use of calibrated instruments such as profilers, hardware counters, and benchmark suites.
5. **Statistical analysis.** Hypothesis testing using t-test, ANOVA, or non-parametric equivalents; reporting effect size and confidence intervals, not just p -values.

Example. Comparing the energy efficiency of three memory allocators in a containerized workload. The independent variable is the allocator; dependent variable is Joules per transaction; controlling variables are CPU frequency, OS, workload trace.

Strengths. High internal validity; results are amenable to replication; quantitative claims are precise.

Limitations. Artificial simplification; threats to ecological validity if the lab environment diverges sharply from production; potentially expensive hardware and software instrumentation.

4.2 Analytical / Theoretical Investigation

Definition. Analytical investigations rely on logical and mathematical reasoning to derive properties of constructs without empirical measurement. They produce proofs, complexity bounds, or formal specifications.

Where it fits in CS. Algorithm analysis, formal verification, cryptography, complexity theory, and theoretical machine learning.

Core Structure.

1. **Statement of claim.** A theorem, conjecture, or formal property.

2. **Assumption set.** Pre-conditions under which the claim holds.
3. **Reasoning apparatus.** Lemmas, proofs by induction, contradiction, or construction.
4. **Bound derivation.** Worst-case, average-case, or amortized complexity expressed asymptotically, e.g., $O(n \log n)$ or $O(n \log n)$.
5. **Counter-example or proof.** Either disproves the claim or establishes it.

Example. Proving that a proposed cache replacement policy is starvation-free using an invariant-based argument over the request queue.

Strengths. Definitive conclusions within the assumption set; immune to measurement noise; foundational.

Limitations. Results may not predict real-world performance; assumes the formal model reflects reality; readers may struggle with notation.

4.3 Simulation-Based Investigation

Definition. A simulation investigation builds a computational model of a system and runs experiments on the model when the real system is inaccessible, unsafe, or too costly to operate at scale.

Where it fits in CS. Network protocol evaluation, distributed systems, large-scale machine learning pipelines, urban computing, and epidemic modeling on graph structures.

Core Structure.

1. **Model formulation.** Identify entities, attributes, and behaviours.
2. **Calibration.** Tune parameters against historical or pilot data.
3. **Validation.** Confirm that the simulator's output matches reference behaviour (face validity, predictive validity).
4. **Scenario execution.** Run synthetic scenarios varying key parameters.
5. **Sensitivity analysis.** Examine how output changes with input perturbation.

Example. Using a discrete-event simulator (e.g., ns-3) to evaluate congestion-control behaviour across 10,000 simulated TCP flows, which would be infeasible on a physical testbed.

Strengths. Scalable, repeatable, safe to explore edge cases.

Limitations. Garbage-in-garbage-out: simulation fidelity is bounded by the model. Publication bias toward optimistic results.

4.4 Survey / Empirical Investigation

Definition. A survey investigation cross-sectionally captures the attitudes, practices, or self-reported behaviours of a population sample, typically through questionnaires or interviews.

Where it fits in CS. Software engineering practice, developer experience, cybersecurity awareness, adoption of programming paradigms.

Core Structure.

1. **Population and sampling.** Define the target population; choose probability or non-probability sampling.
2. **Instrument design.** Question construction, pilot testing, validation (e.g., Cronbach's alpha for reliability, $\alpha \geq 0.7$ typically regarded as acceptable).
3. **Data collection.** Online, telephone, in-person distribution.
4. **Analysis.** Descriptive statistics, factor analysis, regression, or thematic coding for open-ended items.
5. **Reporting.** Response rate, confidence intervals, and discussion of non-response bias.

Example. A study of how Indian IT firms adopted DevOps practices between 2018 and 2024, surveying 250 project managers.

Strengths. Captures breadth; useful for phenomena not directly observable.

Limitations. Self-report bias; cross-sectional nature rules out strong causality.

4.5 Case Study Investigation

Definition. A case study investigates a contemporary phenomenon in its real-world context, relying on multiple sources of evidence, when the boundaries between phenomenon and context are unclear.

Where it fits in CS. Industry pilots, large-scale deployment studies, post-mortem analyses of system failures.

Core Structure.

Following Yin's protocol:

1. **Case definition** (single holistic, single embedded, multiple holistic, multiple embedded).
2. **Case selection** (literal, theoretical, or extreme-case replication logic).
3. **Data triangulation** among interviews, documents, direct observation, and physical artefacts.
4. **Pattern matching** between empirical findings and theoretical propositions.
5. **Case report** with thick description.

Example. Investigating the failure of a multi-region cloud deployment by interviewing SREs, reviewing incident logs, and reproducing the failure in a sandbox.

Strengths. Ecological validity; preserves context; supports theory-building from data.

Limitations. Limited generalizability; researcher subjectivity; replication difficulty.

4.6 Design Science (Constructive) Investigation

Definition. Design science research (DSR) creates and evaluates innovative IT artefacts intended to solve identified problems. The artefact may be a construct, model, method, or instantiation.

Where it fits in CS. HCI prototyping, novel algorithm design, new programming language features, system architectures.

Core Structure. The canonical DSR cycle consists of:

- **Build** — construct an artefact v1.
- **Evaluate** — observe its utility in a realistic setting.
- **Reflect** — refine the artefact into v2.
- **Iterate.**

This maps onto the **Hevner-Chatterjee framework** of three DSR cycles: *relevance cycle* (problem), *rigour cycle* (theory base), *design cycle* (build-evaluate).

Example. Designing a privacy-preserving federated learning aggregator, instantiating it as a prototype, and demonstrating utility on three benchmark datasets.

Strengths. Concrete contribution; immediate applicability; supports cumulative tradition.

Limitations. Requires both engineering rigour and evaluation rigour; risk of conflation between innovation and validation.

4.7 Comparative / Benchmarking Investigation

Definition. A comparative investigation positions a new solution against established baselines on shared workloads and metrics to demonstrate relative merit.

Where it fits in CS. ML model evaluation, database engine comparison, framework-versus-framework studies.

Core Structure.

1. **Selection of baselines.** State-of-the-art and credible baselines, not strawmen.

2. **Standardized workloads.** Public datasets and workloads with documented characteristics.
3. **Common evaluation protocol.** Identical hardware budget, identical preprocessing.
4. **Multiple metrics.** Beyond accuracy alone — include latency, fairness, energy.
5. **Statistical rigour.** Multiple seeds, paired tests, and reporting of variance.

Example. Benchmarking a new transformer-based recommender against four published baselines on three datasets, reporting nDCG@10, MRR, and inference latency.

Strengths. Direct evidence of improvement; reproducible.

Limitations. Risk of "benchmark overfitting"; metric gaming.

4.8 Mixed-Methods Investigation

Definition. A mixed-methods investigation strategically combines qualitative and quantitative strands so that the strengths of one offset the weaknesses of the other.

Where it fits in CS. Increasingly the norm for industry-academic collaborations, where qualitative insights into developer experience complement quantitative performance measurements.

Core Structures. Creswell and Plano Clark identify four major designs:

- **Convergent parallel** — quantitative and qualitative strands run independently, then merge.
- **Explanatory sequential** — quantitative first, qualitative deepens the findings.
- **Exploratory sequential** — qualitative first, quantitative generalizes.
- **Embedded / nested** — one strand plays a supportive role inside the other.

Example. Quantitatively benchmarking a new IDE plugin for refactoring, followed by qualitative interviews with five developers to interpret usage patterns.

Strengths. Triangulation, richer findings.

Limitations. Demands dual competence; longer timelines.

5. Criteria for Selecting an Investigative Approach

The choice is rarely mechanical, but the following decision rubric (Table 5.1) provides a useful starting point.

Table 5.1: Decision rubric for approach selection

Criterion	Favours Experimental	Favours Analytical	Favours Simulation	Favours Survey	Favours Case Study	Favours DSR
Is the real system available?	Yes	n/a	No / expensive	n/a	Yes, in situ	Prototype only
Is a causal claim required?	Strongly yes	Modelled via proof	Modelled in abstract	No	No (rich description)	Limited
Is the construct novel?	Optional	Optional	Optional	n/a	n/a	Yes
Is the population broad?	Small sample	n/a	n/a	Large	One unit or few	n/a
Is generalizability a goal?	Yes (statistical)	High within model	Conditional on simulator fidelity	Statistical	Analytical generalization	Domain-general transfer

Additional pragmatic filters include:

- **Resource envelope:** compute budget, time, human participants.
- **Skill mix:** statistical literacy, proof-writing capability, fieldwork access.
- **Ethical clearance:** human-subject protocols, data privacy.
- **Publication venue norms:** theory venues favour analytical; systems venues favour experimental/DSR; software engineering venues favour case + survey.

6. Instrumentation Across Approaches

Instrumentation is the operational expression of measurement and is tightly coupled to the investigative approach.

- **Experimental.** Profilers (perf, VTune), hardware performance counters, automated test harnesses, benchmark suites (SPEC CPU, MLPerf).
- **Analytical.** Pen-and-paper or proof assistants (Coq, Isabelle, Lean); complexity analyzers.
- **Simulation.** Simulators (ns-3, OMNeT++, SUMO), random workload generators.

- **Survey.** Online survey platforms (Qualtrics, LimeSurvey), validated psychometric instruments.
- **Case study.** Interview protocols, archival templates, logging infrastructure.
- **DSR.** Version-controlled prototype repositories, automated CI pipelines, evaluation harnesses.
- **Comparative.** Reproducibility scripts, Docker containers, compute-tracking sheets.
- **Mixed-methods.** Survey platform + interview transcription tools + statistical package + qualitative coding software (NVivo, ATLAS.ti).

Calibration and validation of instrumentation are mandatory. An uncalibrated clock, an unvalidated rubric, or an unseeded RNG degrades evidence quality irrespective of the approach.

7. Worked Examples

7.1 Example A — Algorithm Research

Problem. Is a proposed in-memory join algorithm faster than the radix-join baseline across varied cardinality distributions?

Approach. Experimental with embedded analytical reasoning (asymptotic bounds) and comparative benchmarking.

Instrumentation. Pin/Intel VTune for cycle counts; Docker containers for reproducibility.

Analysis. Welch's t-test for unequal variances; effect size reported via Cohen's *dd*; family-wise error controlled with Holm-Bonferroni.

7.2 Example B — Usable Security Research

Problem. Do software developers detect insecure code snippets faster using a static-analysis warning redesigned with cognitive load principles?

Approach. Mixed-methods: between-subjects experiment (quantitative) followed by think-aloud interviews (qualitative).

Instrumentation. Eye-tracker, screen recorder, code-review task suite; interview protocol and coding scheme.

Analysis. ANOVA on time-to-detection; thematic analysis of interview transcripts to explain observed patterns.

7.3 Example C — Distributed Systems Research

Problem. Can a leaderless consensus protocol guarantee liveness under bounded latency partitions?

Approach. Analytical proof of liveness combined with simulation exploring the latency threshold.

Instrumentation. TLA+ / Isabelle for proof; custom Java simulator for parameter sweeps.

Analysis. Contradiction-based proof; regression curves from simulation.

8. Common Errors in Choosing an Approach

1. **Tool-led selection.** Picking a survey because Qualtrics is easy, rather than because a survey answers the research question.
 2. **Single-method tunnel vision.** Treating one approach as universally applicable to all sub-questions.
 3. **Confusing method with approach.** Running a questionnaire is a *method*, not an *approach*; the approach decides what kind of evidence is sought.
 4. **Ignoring ecological validity.** A clean lab experiment that fails to reflect deployment realities.
 5. **Neglecting rigour expectations of the venue.** A theory paper without proofs, a systems paper without statistical comparison.
 6. **Over-claiming beyond the approach.** Drawing causal conclusions from cross-sectional survey data.
 7. **No instrumentation validation.** Trusting an ad-hoc timer or unverified rubric.
-

9. Connecting Approach to Downstream Design

The investigative approach is not a stand-alone decision. It cascades into:

- **Data collection design** — what to record, at what granularity, over what time horizon.
- **Data analysis plan** — descriptive, inferential, qualitative coding, or hybrid.
- **Interpretation strategy** — confirmatory, exploratory, or theory-driven.
- **Reporting format** — replication package, open dataset, qualitative codebook.

A useful self-check after approach selection is to write a one-sentence statement of the form:

"Using [approach], with [instrumentation], on [sample/system], I will measure [metric/phenomenon] to test [hypothesis/proposition] under [conditions]." If this statement cannot be completed crisply, the approach choice requires revisiting.

10. Summary

Investigative approaches in computer science research are plural by necessity. Experimental, analytical, simulation-based, survey, case study, design science, comparative, and mixed-methods investigations each contribute a distinct evidentiary logic. The rigorous researcher does not ask "which approach is best?" but rather "which approach is justified *for this problem, given these constraints, to support these claims?*" Selection criteria include the nature of the construct, accessibility of the system, causality requirements, resource envelope, and venue expectations. The chosen approach then orchestrates downstream decisions in data collection, instrumentation, analysis, and interpretation. Mastery of these approaches is a prerequisite for the second unit of this course, which elaborates the data-collection and analysis methods aligned to each approach.

11. Exam Preparation Tips

1. **Frame answers around claims and evidence.** Examiners reward students who explicitly link the research question to the type of evidence required.
 2. **Master the strengths-limitation trade-off.** Mere listing of approaches without acknowledging their vulnerabilities costs marks.
 3. **Be CS-specific in examples.** Generic "social science" or "physics" illustrations signal shallow engagement with the discipline.
 4. **Use the terminology precisely.** Distinguish *approach, method, technique, and instrument*; many rubric points hinge on this.
 5. **Practise method-justification writing.** Two-paragraph justifications are common short-answer questions.
 6. **Memorize one canonical CS study per approach.** A real study (author, year, venue, finding) is far more convincing than an abstract description.
 7. **Prepare for scenario questions.** "Given problem X and constraints Y, which approach would you choose and why?" tests synthesis, not recall.
 8. **Revisit validity terminology.** Internal, external, construct, and conclusion validity recur across question papers.
 9. **Be ready to draw a comparison table.** Examiners often ask for tabular contrasts between two approaches.
 10. **Plan time on diagrams.** A simple block diagram of, say, the DSR cycle, can earn quick marks if drawn cleanly.
-

12. Practice Questions

Short Answer (5–6 marks each)

1. Differentiate between *investigative approach* and *research method* with a CS example.
2. Under what conditions would a simulation-based investigation be preferable to an experimental one? Illustrate with an example from networking or distributed systems.
3. Explain the role of the *relevance*, *rigour*, and *design* cycles in the Hevner-Chatterjee design science framework.
4. Why are cross-sectional surveys unable, on their own, to establish causality in software-engineering research?

Medium Answer (8–10 marks each)

5. Compare and contrast experimental and case-study investigations along six dimensions of your choice. Conclude with the kind of CS problem best suited to each.
6. Describe the principal threats to validity in a controlled experiment measuring the runtime of a new database index. Propose mitigations.
7. For the problem "*How do Indian product-based startups evaluate the cost-effectiveness of migrating monoliths to microservices?*", propose an investigative approach and justify it with reference to validity, resources, and venue expectations.

Long Answer / Project (15–20 marks)

8. Design a mixed-methods investigation for a CS research problem of your choice. Include: research questions, approach diagram, instrumentation plan, sampling strategy, analysis plan, threats to validity, and a reproducibility statement.
9. Critically review a published CS paper (provided by the instructor) for the appropriateness of its investigative approach. Highlight mismatches, missing evidence, and recommended revisions.
10. Construct a decision rubric (table or flowchart) that maps a CS research problem to an investigative approach, and apply it to three diverse problems from algorithmic, systems, and HCI sub-fields.

13. Suggested Readings

- Wohlin, C., Runeson, P., Höst, M., Ohlsson, M. C., Regnell, B., & Wesslén, A. (2012). *Experimentation in Software Engineering*. Springer.

- Yin, R. K. (2018). *Case Study Research and Applications: Design and Methods* (6th ed.). SAGE.
 - Hevner, A., & Chatterjee, S. (2020). Design Science Research in Information Systems. In *Design Research in Information Systems*. Springer.
 - Creswell, J. W., & Plano Clark, V. L. (2018). *Designing and Conducting Mixed Methods Research* (3rd ed.). SAGE.
 - Kitchenham, B., & Charters, S. (2007). *Guidelines for performing Systematic Literature Reviews in Software Engineering*. EBSE Technical Report.
 - ACM Code of Ethics and Professional Conduct (current edition).
-

End of Lecture 5 Notes